

Fundamental File Structure Concepts

Outline

- Field and record organization
- Sequential search and direct access
- Sequential Files
- Sorted Sequential Files
- Co-sequential processing

Files

A file can be seen as

1. A stream of bytes (no structure), or
2. A collection of records with fields

A Stream File

- File is viewed as a sequence of bytes:

```
87359CarrollAlice in wonderland38180FolkFile Structures ...
```

- Data semantics is lost: there is no way to get it apart again.

Field and Record Organization

Definitions

Record: a collection of related fields.

Field: the smallest logically meaningful unit of information in a file.

Key: a subset of the fields in a record used to identify (uniquely) the record.

e.g. In the example file of books:

- Each line corresponds to a record.
- Fields in each record: ISBN, Author, Title

Record Keys

- **Primary key:** a key that uniquely identifies a record.
- **Secondary key:** other keys that may be used for search
 - Author name
 - Book title
 - Author name + book title
- Note that in general not every field is a key (keys correspond to fields, or a combination of fields, that may be used in a search).

Field Structures

- Fixed-length fields

87359Carroll Alice in wonderland

38180Folk File Structures

- Begin each field with a length indicator

058735907Carroll19Alice in wonderland

053818004Folk15File Structures

- Place a delimiter at the end of each field

87359|Carroll|Alice in wonderland|

38180|Folk|File Structures|

- Store field as keyword = value

ISBN=87359|AU=Carroll|TI=Alice in wonderland|

ISBN=38180|AU=Folk|TI=File Structures

Type	Advantages	Disadvantages
Fixed	Easy to read/write	Waste space with padding
Length-based	Easy to jump ahead to the end of the field	Long fields require more than 1 byte
Delimited	May waste less space than with length-based	Have to check every byte of field against the delimiter
Keyword	Fields are self describing. Allows for missing fields	Waste space with keywords

Record Structures

1. Fixed-length records.
2. Fixed number of fields.
3. Begin each record with a length indicator.
4. Use an index to keep track of addresses.
5. Place a delimiter at the end of the record.

Fixed-length records

Two ways of making fixed-length records:

1. Fixed-length records with fixed-length fields.

87359	Carroll	Alice in wonderland
03818	Folk	File Structures

2. Fixed-length records with variable-length fields.

87359	Carroll	Alice in wonderland	<i>unused</i>
38180	Folk	File Structures	<i>unused</i>

Variable-length records

- Fixed number of fields:

```
87359|Carroll|Alice in wonderland|38180|Folk|File Structures| ...
```

- Record beginning with length indicator:

```
3387359|Carroll|Alice in wonderland|2638180|Folk|File Structures| ..
```

- Use an index file to keep track of record addresses:
 - The index file keeps the byte offset for each record; this allows us to search the index (which have fixed length records) in order to discover the beginning of the record.
- Placing a delimiter: e.g. end-of-line char

Type	Advantages	Disadvantages
Fixed length record	Easy to jump to the i-th record	Waste space with padding
Variable-length record	Saves space when record sizes are diverse	Cannot jump to the i-th record, unless through an index file

File Organization

- Four basic types of organization:
 1. **Sequential** ← today
 2. Indexed
 3. Indexed Sequential
 4. Hashed
- In all cases we view a file as a sequence of **records**.
- A record is a list of **fields**. Each field has a data type.

File Operations

- Typical Operations:
 - Retrieve a record
 - Insert a record
 - Delete a record
 - Modify a field of a record
- In direct files:
 - Get a record with a given field value
- In sequential files:
 - Get the next record

Sequential (Pile) files

- Records are stored contiguously on the storage device.
- Sequential files are read from beginning to end.
- Some operations are very efficient on sequential files (e.g. finding averages)
- Organization of records:
 1. Unordered sequential files (**pile files**)
 2. **Sorted** sequential files (records are ordered by some field)

Pile Files

- A pile file is a succession of records, simply placed one after another with no additional structure.
- Records may vary in length.
- Typical Request:
 - Print all records with a given field value
 - e.g. print all books by *Folk*.
 - We must examine each record in the file, in order, starting from the first record.

Searching Sequential Files

- To look-up a record, given the value of one or more of its fields, we must search the whole file.
- In general, (***b*** is the total number of blocks in file):
 - At least 1 block is accessed
 - At most *b* blocks are accessed.
 - On average $1/b * b(b + 1) / 2 \Rightarrow b/2$
- Thus, time to find and read a record in a pile file is approximately :

$$T_F = s + r + (b / 2) * btt$$



Time to fetch one record

Nomenclature

- s : average seek time
- r : average rotational delay
- btt : transfer time (time required to transfer 1 block of data)
- b : number of blocks in the file
- Bfr : Blocking factor (number of records in a block)
- B : size of a block
- R : size of a record

Exhaustive Reading of the File

- Read and process all records (reading order is not important)

$$T_X = s + r + b * btt$$

(approximately twice the time to fetch one record)

- e.g. Finding averages, min or max, or sum.
 - Pile file is the best organization for this kind of operations.
 - They can be calculated using double buffering as we read though the file once.

Inserting a new record

- Just place the new record at the end of the file (assuming that we don't worry about duplicates)
 - Read the last block
 - Put the record at the end.

$$\Rightarrow T_I = s + r + btt + \underbrace{(2r - btt)}_{\text{Time to wait}} + btt$$

$$\Rightarrow T_I = s + r + btt + 2r$$

- Q) What if the last block is full?

Updating a record

- For fixed length records:

$$T_U (\text{fixed length}) = T_F + 2r$$

- For variable length records update is treated as a combination of delete and insert.

$$T_U (\text{variable length}) = T_D + T_I$$

Deleting Records

- Operations like create a file, add records to a file and modify a record can be performed physically by using basic file operations (open, seek, write, etc)
- What happens if records are deleted? There is no basic operation that allows us to “remove part of a file”.
- Record deletion should be taken care by the program responsible for file organization.

Strategies for record deletion

1. Record deletion and Storage compaction:

- Deletion can be done by “marking” a record as deleted.
 - e.g. Place ‘*’ at the beginning of the record
- The space for the record is not released, but the program must include logic that checks if record is deleted or not.
- After a lot of records have been deleted, a special program is used to squeeze the file – this is called **Storage Compaction**.

Strategies for record deletion (cont.)

2. Deleting fixed-length records and reclaiming space dynamically.

- How to use the space of deleted records for storing records that are added later?
 - Use an “AVAIL LIST”, a linked list of available records.
 - A header record (at the beginning of the file) stores the beginning of the AVAIL LIST
 - When a record is deleted, it is marked as deleted and inserted into AVAIL LIST

Strategies for record deletion (cont.)

3. Deleting variable length records

- Use AVAIL LIST as before, but take care of the variable-length difficulties.
- The records in AVAIL LIST must store its size as a field. Exact byte offset must be used.
- Addition of records must find a large enough record in AVAIL LIST

Placement Strategies

- There are several placement strategies for selecting a record in AVAIL LIST when adding a new record:
 1. First-fit Strategy
 - AVAIL LIST is not sorted by size; first record large enough is used for the new record.
 2. Best-fit Strategy
 - List is sorted by size. Smallest record large enough is used.
 3. Worst-fit strategy
 - List is sorted by decreasing order of size; largest record is used; unused space is placed in AVAIL LIST again.

Problem 1

- Estimate the time required to reorganize a file for storage compaction. (Derive a formula in terms of the number of blocks, b , in the original file; b_{tt} ; number of records n in the new file and blocking factor B_{fr})
 - i. Reorganize the file with one disk drive
 - ii. Reorganize the file with two disk drives

Problem 2

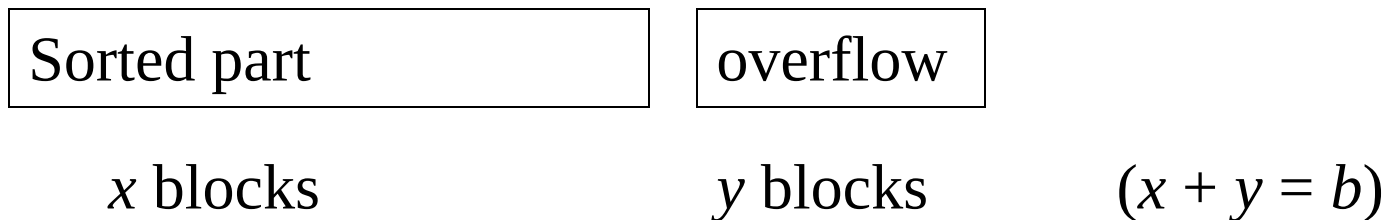
- Given two pile files A and B with $n=100,000$ and $R=400$ bytes each, we want to create an intersection file. Assume that 70% of the records are in common and the available memory for this operation is 10M.
- Calculate a timing estimate for deriving and writing the intersection file (Use $s = 16$ ms, $r = 8.3$ ms, $btt = 0.84$ ms, $B=2400$ bytes.)

Sorted Sequential Files

- Sorted files are usually read sequentially to produce lists, such as mailing lists, invoices.etc.
- A sorted file cannot stay in order after additions (usually it is used as a temporary file).
- A sorted file will have an overflow area of added records. Overflow area is not sorted.
- To find a record:
 - First look at sorted area
 - Then search overflow area
 - If there are too many overflows, the access time degenerates to that of a sequential file.

Searching for a record

- We can do binary search (assuming fixed-length records) in the sorted part.



- Worst case to fetch a record :

$$T_F = (\log_2 x) * (s + r + btt)$$

- If the record is not found, search the overflow area too. Thus total time is:

$$T_F = (\log_2 x) * (s + r + btt) + s + r + (y / 2) * btt$$

Problem 3

- Given the following:
- Block size = 2400 bytes
 - File size = 40MB
 - Block transfer time (btt) = 0.84ms
 - $s = 16$ ms
 - $r = 8.3$ ms

Q1) Calculate T_F for a certain record

- a) in a pile file
- b) in a sorted file (no overflow area)

Q2) Calculate the time to look up 10000 names.

Co-sequential Processing

- Co-sequential processing involves the coordinated processing of two or more sequential files to produce a single output file.
- Two main types of resulting file are:
 - Matching (intersection) of the records in the files.
 - Merging (union) of the records in the files.

Examples of applications

1. Matching:

- i. Finding the intersection file
- ii. Batch Update
 - **Master file** – bank account info (account number, person name, balance) – sorted by account number
 - **Transaction file** – updates on accounts (account number, credit/debit info) – sorted by account number

2. Merging:

- i. Merging two class lists keeping alphabetic order.
- ii. Sorting large files (break into small pieces, sort each piece and then merge them)

Problem 4: Intersection file

- Solve problem 3 with two sorted files.

Algorithm for Co-sequential Batch Update

1. initialize pointers to the first records in the master file and transaction file.
2. Do until pointers reach end of file:
 - i. Compare keys of the current records in both files
 - ii. Take appropriate action *
 - iii. Advance one (or both) of the pointers.

Appropriate Action

if master key < transaction key

- copy master file record to the end of the new master file.
- advance master file pointer.

if master key > transaction key

- if transaction is an insert
copy transaction file record to the end of new master file
else log an error
- advance the transaction file pointer.

if master key = transaction key

- If transaction is modify
copy modified master file record to the end of the new file
- If transaction is insert log an error
- If transaction is delete, do nothing
- In all three cases, advance both the master and transaction file pointer